



0. Python: High-Level Interpreted Programming Language

"There should be one—and preferably only one—obvious way to do it."

Contents:

1. What is Python?

Python is an *Interpreted* Language

Python is a *High-Level* Language

Python is *Object-Oriented*

The History of Python

Python is Open Source

Python is a General-Purpose Language

Python in the Browser — PyScript

Python's Origins and Influence

Major Python Versions

Recommended Python Version

2. Tools for Python

Thonny — The Beginner-Friendly Workshop

PyCharm — The Professional Python Environment

Google Colab — Python in the Cloud

VS Code — The Universal Tool

Online Compilers — Fast but Limited

3. Python Courses and Learning Resources

GeeksforGeeks — The Encyclopedia for Learners

[DataCamp — Learning Through Practice](#)

[W3Schools and Programmiz — Simple and Accessible Tutorials](#)

[YouTube Tutorials — Free and Massive](#)

[Summary](#)

[References](#)

To most people, Python is just a snake. But in computer science, engineering, artificial intelligence, cybersecurity, data science, and software development, etc. Python is something entirely different. It is one of the most widely used programming languages in the world.

In many ways, Python became the “English language” of programming — not because it is the fastest, or the most powerful in every category, but because almost everyone can read it, learn it, and use it.

And before we properly understand Python itself, we first need to step back and understand something much more fundamental: What does it actually mean for a programming language to be *low-level* or *high-level*?

Because computers, by nature, do not understand human language. They understand electrical signals. Binary. Instructions. Machine code.

Concretely, it only understands something like: `10101100 00101101`. To a computer, that makes perfect sense. To humans? It looks unreadable. Trying to directly communicate with a computer in pure machine code is like trying to explain English literature to an alien species using blinking lights.

And that is exactly why programming languages were created. They act as translators between humans and machines. Low-level languages stay closer to the computer. They focus on performance, memory control, and hardware interaction. Examples include: Assembly and Machine Code. These are extremely fast, but difficult for humans to work with.

So eventually, programming evolved toward something more human-readable: High-level programming languages. Languages designed not for machines first, but for people. And within high-level languages, I personally like dividing them into three major categories:

- **Compiled Languages:** These are languages that are transformed into machine code *before* execution. Examples are C, C++, C#, Go, Rust. They

are usually fast, efficient, and heavily used in systems programming, game engines, operating systems, and performance-critical software.

- **Interpreted Languages:** These languages are executed more dynamically through an interpreter. Examples are Python, JavaScript, Ruby, PHP. They are often easier to write, easier to debug, and much faster to develop with.
- **Hybrid Languages:** Some languages sit between both worlds. The most famous example is Java. Java first compiles into bytecode, then runs through the Java Virtual Machine (JVM), making it something of a hybrid approach.

And Python belongs to the High-Level Interpreted group. But here is what makes Python special. Python was designed around one major philosophy: ***Code should be readable by humans first.*** That is why Python syntax often feels surprisingly close to natural language.

For example:

```
if temperature > 30:  
    print("It is hot today")
```

Even someone who has never programmed before can roughly understand what this code is trying to do. And that accessibility changed everything.

Python became one of the most important languages in: Artificial Intelligence, Machine Learning, Data Science, Automation, Cybersecurity, Web Development, Scientific Computing, Robotics, Education. Literally everything!! Because instead of fighting the language, people could focus on solving problems.

And that is the reason Python became so influential. Not because it tries to be the most complex language. But because it removes complexity between humans and computation.

So before we even start writing code, understand this: Python is not just a programming language. It is a bridge between human logic, and machine execution.

1. What is Python?

Now that we understand the difference between low-level and high-level programming languages, we can finally answer the main question: ***What exactly is Python?***

Python is *an interpreted, high-level, object-oriented programming language* designed to make programming more readable, flexible, and accessible for humans.

That sentence sounds simple, but every part of it matters. So let's slowly break it apart.

Python is an *Interpreted* Language

When we say Python is interpreted, it means Python code is executed line by line through an interpreter instead of being fully translated into machine code beforehand.

Think of it like this:

- A compiled language is similar to translating an entire book before publishing it.
- An interpreted language is more like having a live translator speaking sentence by sentence in real time.

This gives Python several advantages: Faster development. Easier debugging. More flexibility. Simpler experimentation. Which is exactly why Python became extremely popular in various fields. Because researchers and developers often care more about solving problems quickly than squeezing out every last bit of hardware performance.

Python is a *High-Level* Language

High-level means Python is designed to be understandable by humans, not just computers. Instead of forcing programmers to work with complex memory management, binary instructions, or hardware-level operations, Python abstracts most of that complexity away.

For example, compare this idea:

```
name = "Iris"
print(name)
```

Even someone new to programming can roughly understand what this code does. Python focuses heavily on readability and simplicity. In fact, one of Python's core philosophies is: ***"Code is read more often than it is written"***. And that philosophy shaped the entire language.

Python is *Object-Oriented*

Object-oriented programming (OOP) is a programming style centered around objects. An object can represent almost anything: A student, a car, a bank account, a machine learning model, even a game character. Each object contains: Data (attributes) and Behaviors (methods/functions)

For example:

```
class Student:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello")

student1 = Student("Iris")
student1.greet()
```

Here:

- `Student` is an object blueprint (class)
- `student1` is an object
- `name` is data
- `greet()` is behavior

Object-oriented programming helps developers organize large programs into structured, reusable systems. And Python supports this extremely well.

The History of Python

Python was created by Guido van Rossum in 1989. Its official public release came in 1991.

Interestingly, Python was designed with one major goal: ***Make programming simpler and more readable***. And unlike many languages that prioritize complexity or strict syntax, Python intentionally tries to reduce friction between humans and code.

Even the name "Python" did not come from the snake. It came from the British comedy group Monty Python, which Guido van Rossum enjoyed watching.

Python is Open Source

Python is completely open-source. That means: Anyone can use it. Anyone can modify it. Anyone can distribute it. This openness helped Python spread globally across universities, companies, researchers, startups, and communities.

Today, millions of developers contribute to Python libraries, frameworks, and tools. And because of that, Python has one of the largest ecosystems in programming.

Python is a General-Purpose Language

Python is called a general-purpose programming language because it can be used for almost anything.

You can build: Websites, AI systems, Machine Learning models, Desktop applications, Games, Robotics systems, Data analysis pipelines, Automation scripts, Cybersecurity tools. Scientific simulations. And this versatility is one of Python's biggest strengths.

Python is currently used in:

- Google search systems
- NASA mission-critical projects

- Financial transaction systems at the New York Stock Exchange

Some of the biggest companies using Python include: Google, Dropbox, Mozilla, IBM, Cisco, Qualcomm, Quora, HP. That alone tells you something important: Python is not just a beginner language. It is an industry language.

Python in the Browser — PyScript

For many years, Python mainly ran on local machines or servers. But in 2022, the Anaconda ecosystem introduced PyScript. PyScript allows Python code to run directly inside web browsers — something traditionally dominated by JavaScript.

This opened new possibilities for: Interactive web applications, Browser-based data science, Educational tools, Lightweight AI demonstrations.

In many ways, PyScript attempts to bring Python closer to the web ecosystem.

Python's Origins and Influence

Python did not appear from nowhere. It was heavily influenced by earlier languages such as: Pascal, C, and C++. You can actually see traces of these languages in Python's design philosophy and syntax structure.

But Python also inspired many modern languages afterward. Languages influenced by Python include: Ruby, Go, Swift. Especially Ruby, which many developers jokingly describe as: *"Another version of Python with a different philosophy"*.

Major Python Versions

Python evolved through multiple generations.

- **Python 2.x:** Python 2 was extremely popular for many years. However, it is now considered outdated and officially unsupported.
- **Python 3.x:** Python 3 is the modern standard. It introduced many improvements: Better Unicode support, cleaner syntax, Improved libraries, better consistency

Today, almost all modern Python development uses Python 3.

Recommended Python Version

As of 2025, the latest version is: **Python 3.14.0**

However, newer does not always mean better for learning. Some libraries may lag behind the newest releases. So while older Python 3.x versions still work perfectly fine, my strong recommendation for learning and development is: **Python 3.12.x**.

Because it is modern, stable, well-supported, and highly compatible with most libraries used in: Machine Learning, AI, Data Science, Automation, Web Development. And for beginners, stability matters more than chasing the newest version.

So when people ask: “What is Python?”. The technical answer is simple: Python is an interpreted, high-level, object-oriented, general-purpose programming language. But the practical answer is much more interesting: **Python is a language designed to make humans focus less on syntax, and more on ideas**. And that design philosophy is exactly why Python became one of the most important programming languages in modern computing.

2. Tools for Python

Now that we understand what Python is... we need somewhere to actually write it. Because programming is not done in theory alone. Eventually, every programmer reaches the same point: **“Alright... where do I type the code?”**.

And this is where development tools come in. When working with Python, we usually use something called an IDE or a code editor.

Think of them as workshops for programmers. They help us: Write code; Run programs; Debug errors; Organize projects; Install libraries; Build applications more efficiently. And depending on your goals, different tools fit different kinds of learners.

Thonny — The Beginner-Friendly Workshop

If you are completely new to programming, then Thonny is honestly one of the best places to start. Why? Because Thonny was designed specifically for beginners. The interface is simple. Minimal. Clean. It avoids overwhelming learners with too many professional-level tools at once.

And one of its best features is something beginners desperately need: A very easy debugger. You can literally watch your code execute line by line and see variables changing in real time. That is incredibly useful when learning programming logic.

So if Python feels intimidating at first, Thonny makes it feel approachable.

Install Thonny: <https://thonny.org/>

PyCharm — The Professional Python Environment

Then we move toward something much more powerful: PyCharm. PyCharm is considered one of the most advanced IDEs for Python development.

Unlike beginner-focused tools, PyCharm is built for large projects, professional development, backend systems, data science, machine learning, web applications.

It comes packed with features: Intelligent code suggestions; Advanced debugging; Project management; Virtual environment integration; Git support; Database tools

At first, PyCharm may feel heavy or complicated. But once you start building serious Python projects, you begin to understand why developers love it. It is less like a notebook, and more like an entire engineering laboratory.

Install PyCharm IDE: <https://www.jetbrains.com/pycharm/>

Google Colab — Python in the Cloud

Sometimes, installing things locally becomes annoying. Especially in Machine Learning. Different library versions. GPU setup. CUDA compatibility. Dependency conflicts.

And that is exactly why Google Colab became so popular. Google Colab allows you to write and run Python directly inside your browser. No setup required.

And more importantly, it gives access to cloud GPUs for AI and Deep Learning experiments. Which means even people with weaker laptops can still train Machine Learning models.

Colab is heavily used in: AI, Machine Learning, Deep Learning, Data Science, Research, and Education

You can think of it as: Jupyter Notebook, but hosted by Google. And honestly, for learning Machine Learning, it is one of the most convenient tools available.

Run Google Colab: <https://colab.research.google.com/>

VS Code — The Universal Tool

Next comes one of the most popular development tools in modern programming: Visual Studio Code. Usually shortened as VS Code..

Unlike PyCharm, VS Code is not only for Python. It supports almost every major programming language through extensions. That flexibility made it incredibly popular.

You can use VS Code for: Python, Web Development, AI, Cybersecurity, Cloud Computing, Game Development, DevOps, Mobile Development, and much more.

Its biggest strengths are: Lightweight performance. Massive extension ecosystem. Customizability. Integrated terminal. Git integration

VS Code is like a customizable workstation. You shape it into whatever kind of environment you need.

Download Visual Studio Code: <https://code.visualstudio.com/download>

Online Compilers — Fast but Limited

There are also online Python compilers and interpreters available directly through websites. These are useful for very small programs, quick testing, practicing syntax, running simple exercises

However, I usually only recommend them for beginner-level code. Why? Because online compilers often struggle with external libraries, file handling,

complex environments, large projects, hardware acceleration, package installation.

So while they are convenient. They are not ideal for serious development. Think of them as calculators. Useful for quick experiments. But not where you build full systems.

Some compilers I found on the internet:

<https://www.programiz.com/python-programming/online-compiler/>

<https://www.online-python.com/>

3. Python Courses and Learning Resources

Now here comes the important question:

| ***“Where should I actually learn Python? Apart from... this course??”***

And honestly, there is no single perfect source. Different platforms teach differently. Some focus on theory. Some focus on projects. Some focus on exercises. Some focus on speed. But here are some of the most useful learning resources for beginners.

GeeksforGeeks — The Encyclopedia for Learners

GeeksforGeeks is one of the most famous programming education platforms on the internet.

I sometimes jokingly call it: ***“Wikipedia for programmers”***. Because whenever beginners encounter a concept they do not understand, there is a very high chance GeeksforGeeks already has an article about it.

It contains: Python tutorials, Data structures, Algorithms, Machine learning, Interview preparation, Programming exercises. And one of its biggest strengths is accessibility. Most articles are beginner-friendly.

DataCamp — Learning Through Practice

DataCamp focuses heavily on interactive learning. Especially in: Python, Data Science, Machine Learning, AI, and Analytics

Instead of only reading theory, learners actively write code while learning. That hands-on style is extremely effective. Because programming is not something you truly learn by watching alone. You learn it by writing code. Breaking code. Fixing code. Again and again.

W3Schools and Programmiz — Simple and Accessible Tutorials

Two very beginner-friendly tutorial platforms are: W3Schools and Programmiz

These are excellent for quick explanations, syntax references, small examples, basic exercises

Their tutorials are usually straightforward and easy to follow. Especially for people completely new to programming.

YouTube Tutorials — Free and Massive

And of course, there is YouTube. Platforms like YouTube contain thousands of Python tutorials ranging from beginner level to advanced AI engineering.

You can find crash courses, full university lectures, project tutorials, data science walkthroughs, AI implementation guides, web development tutorials

But there is something important to remember: Not all tutorials are equally good. Some teach concepts clearly. Others only teach copying code. So while YouTube is powerful, you still need critical thinking while learning from it.

Summary

This entire section is only an introduction. Nothing more. And honestly, every course begins this way. Before building systems, writing algorithms, or training AI models. We first need foundations.

So in this section, we explored the basic world surrounding Python:

- What Python actually is
- Why high-level languages exist

- The difference between interpreted and compiled languages
- Python's philosophy and history
- The tools used to write Python code
- The platforms available for learning it

And while none of this may feel "advanced" yet. It matters more than people think. Because beginners often rush directly into writing code without understanding the ecosystem around the language itself.

But programming is not only about syntax. It is also about tools. Workflows. Communities. Resources. Ways of thinking. And Python became powerful not just because of the language itself. But because of the enormous ecosystem built around it.

So for now, do not worry about mastering everything immediately. At this stage, your goal is simply to become familiar with the environment. Understand the landscape. Know the tools. Know where to learn. Know what Python is trying to achieve.

Because in the next sections, we finally begin writing actual Python code.

References

<https://www.python.org/>

<https://www.geeksforgeeks.org/python/history-of-python/>

<https://www.geeksforgeeks.org/python/introduction-to-python/>

Next Section:

</> 1. Python: Basic Syntaxes